

The Strangler Fig Pattern is a gradual software modernization strategy where a new system is built around an old, legacy system (monolith) to incrementally replace its functionality, much like a strangler fig plant envelops and replaces its host tree. An intermediary proxy (facade) intercepts requests, routing them to new services as they're built, allowing old and new components to coexist, reducing risk, and enabling continuous delivery until the monolith can be fully decommissioned. [🔗](#)

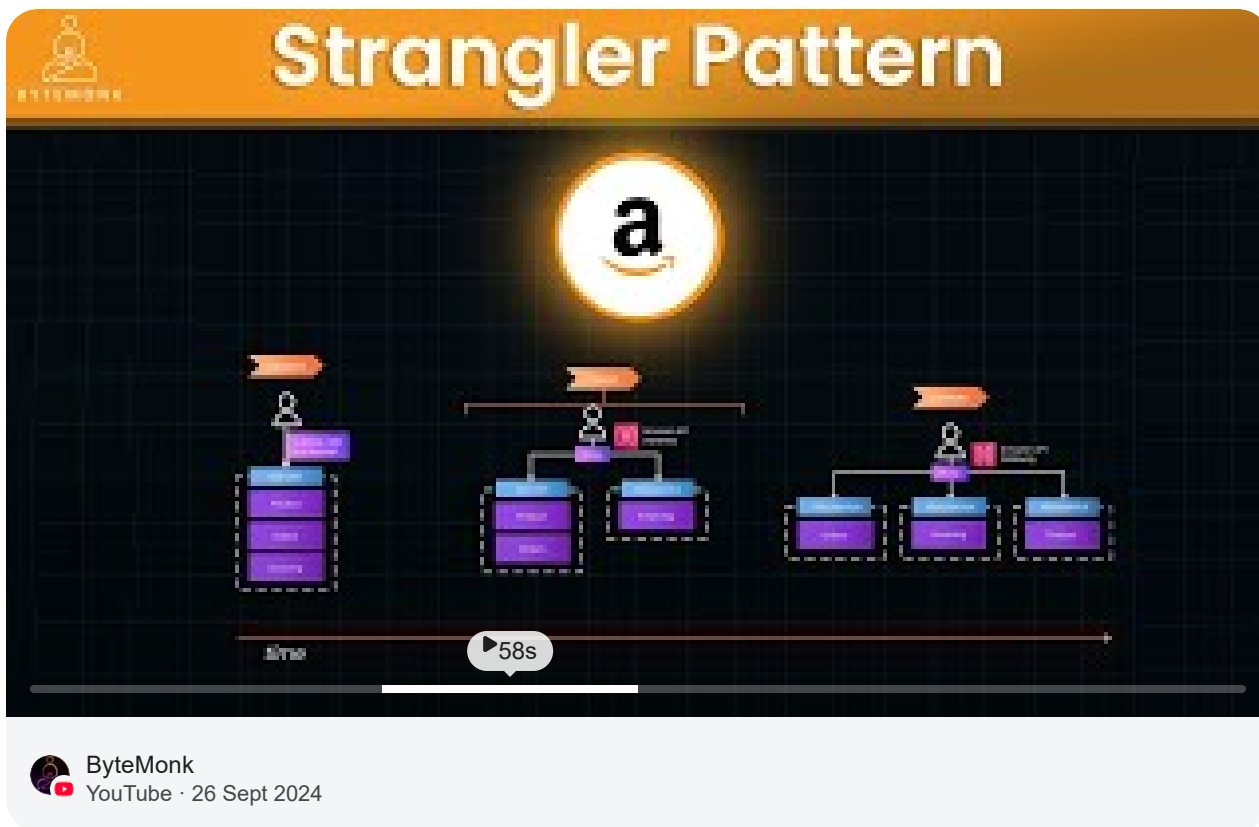
This video explains the concept of the Strangler Fig Pattern:



How it works

1. **Introduce a Facade:** Place a proxy or API Gateway in front of the legacy system to intercept all incoming requests. [🔗](#)
2. **Build New Features Incrementally:** Develop new functionalities as separate services, often as microservices, that run alongside the monolith. [🔗](#)
3. **Redirect Traffic:** Configure the facade to route specific requests to the new services instead of the old system. [🔗](#)
4. **Eliminate Old Code:** As more functionality moves to the new system, the corresponding parts of the legacy monolith become obsolete and can be removed. [🔗](#)
5. **Repeat:** Continue this process until the entire legacy system is "strangled" and replaced. [🔗](#)

You can watch this video to see how to implement the Strangler Pattern:



Key benefits

- **Reduced Risk:** Avoids large, risky "big bang" rewrites by allowing gradual, tested changes. [🔗](#)
- **Improved Business Continuity:** The legacy system remains functional, ensuring minimal disruption during migration. [🔗](#)
- **Continuous Delivery:** Enables ongoing feature development and delivery while modernizing. [🔗](#)
- **Incremental Value:** Delivers value sooner by replacing critical features first. [🔗](#)

When to use it

- Modernizing monolithic applications, especially into microservices.
- When a complete rewrite is too risky or complex.
- To improve scalability, maintainability, or adopt new technologies.